Design Document

AI-Powered Telecom Network Operations Assistant
Problem, architecture, technology choices, retrieval, agents, evaluation, trade-offs (submission deliverable 2)

Item	Value
Domain	Telecom network operations (5G RAN performance and faults)
Solution	Evidence-grounded RAG assistant with six agents, ML risk models and an LLM explanation behind a grounding gate
LLM	NVIDIA API Catalog, openai/gpt-oss-20b (Claude supported as an alternative)
Embeddings / vector DB	all-MiniLM-L6-v2 / ChromaDB (HNSW, cosine)
Frameworks	FastAPI, Streamlit, pandas, scikit-learn, Sentence-Transformers, DeepEval, pytest
Repository	Jerom_Telecom

Contents

Contents 2

1. Problem statement 3

2. Architecture design 3

3. Technology selection 4

4. Retrieval strategy 5

5. Agent design 6

6. Evaluation strategy 7

7. Challenges and trade-offs 7

1. Problem statement

Network operations centre (NOC) engineers face a continuous stream of KPI degradations and alarms. For each one they must decide what kind of problem it is, how severe and urgent it is, what probably caused it and what to check or change — usually by searching past incidents and by experience.

Business problem. Diagnosis is slow and inconsistent, knowledge of past incidents is hard to search, and generic AI chatbots are unsafe here because they invent measurements, incident IDs and confident root causes. A wrong 'fact' in a network decision costs outage time.

Objective. Given a plain-language description (optionally with measured values), return within seconds: similar historical cases as evidence, a derived category, severity and priority, a predicted fault risk next to the observed historical rate, ranked root-cause hypotheses, what to check and what to do — with every statement traceable to its source and nothing invented.

Requirement	How the design meets it
No fabricated data	raw data is the source of truth; the LLM only explains; a grounding gate rejects any number, ID or claim not in the evidence
Explainable labels	categories, severity and priority come from documented YAML rules
Honest uncertainty	hypotheses with heuristic confidence; ML predictions with an 80% interval
Fast and local	runs on a laptop CPU; the system works offline without an LLM

2. Architecture design

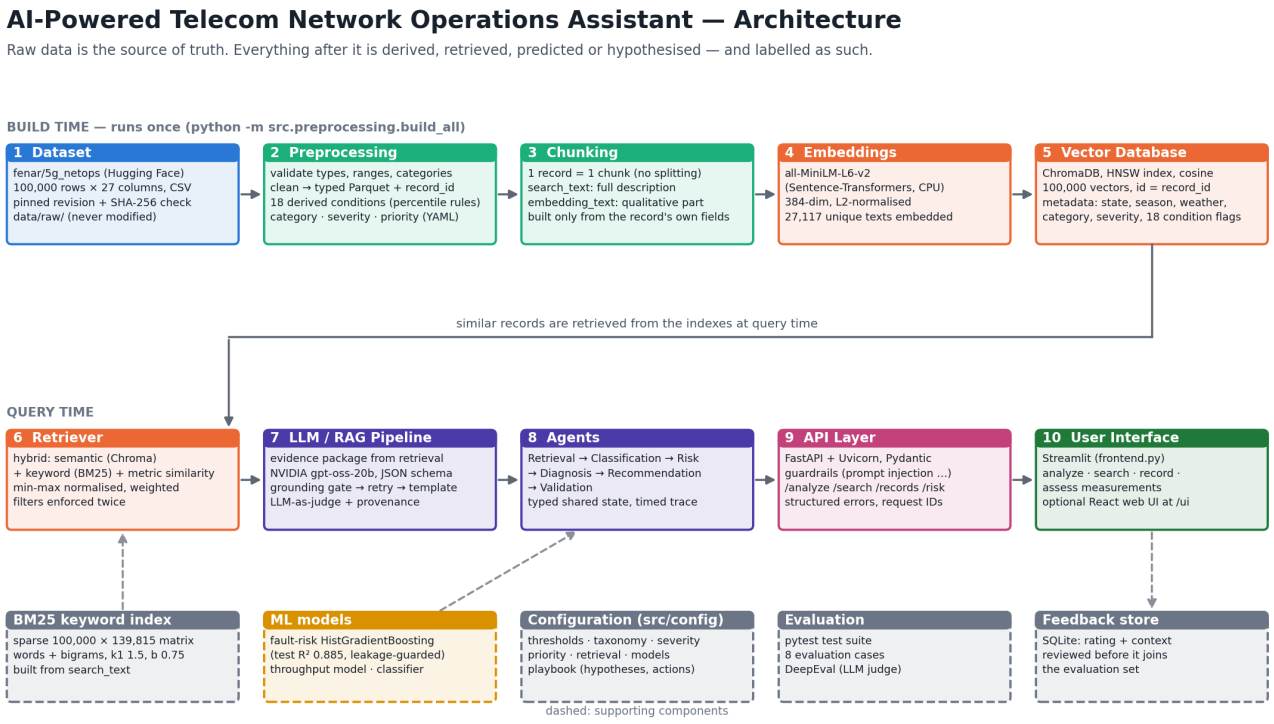


Figure 2.1 — Architecture (architecture/Architecture_Diagram.pdf).

The system has two phases. **Build time** (once): the pinned dataset is validated and cleaned, 18 conditions and the category/severity/priority labels are derived by rules, each record becomes one chunk of text, chunks are embedded into ChromaDB and indexed by BM25, and the ML models are trained. **Query time** (every question): the Streamlit UI calls the FastAPI API; guardrails check the

input; the orchestrator runs six agents that retrieve evidence, classify, predict risk, rank hypotheses, build recommendations with an LLM explanation, and validate the result.

Layer	Responsibility	Code
Preprocessing	download, validate, clean, derive conditions and labels, chunk	`src/preprocessing/`
RAG	embeddings, ChromaDB, BM25, filters, query understanding, hybrid retrieval, pipeline, LLM provider, narrative, grounding, judge	`src/rag/`
Services	classification, risk, root cause, recommendations, evidence, provenance, report, guardrails, feedback, ML models	`src/services/`
Agents	six agents, shared state, orchestrator	`src/agents/`
API	FastAPI routes, Pydantic schemas, structured errors	`src/api/, src/main.py`
UI	Streamlit front end; optional React web UI	`frontend.py, web/`
Configuration	every threshold, weight and rule	`src/config/*.yaml`

Direction of trust. Data flows only from the raw file upwards. Each statement in an answer carries one of seven provenance types — user input, raw data, derived feature, historical evidence, ML prediction, engineering hypothesis, recommendation — so a measured fact can never be confused with a prediction or a guess.

3. Technology selection

Component	Choice	Why	Alternatives considered
LLM	NVIDIA API Catalog, openai/gpt-oss-20b (temperature 0.2, reasoning effort low)	OpenAI-compatible endpoint with JSON-schema structured output; open-weight model; low reasoning effort measured fastest (5–16 s) with clean grounding; Claude can be selected by configuration	Claude haiku-4.5 (used earlier, supported); a local LLM (too slow on CPU)
Embedding model	sentence-transformers/all-MiniLM-L6-v2 (384-dim)	small, fast on CPU, strong general-purpose sentence similarity; chunks are short qualitative texts	larger models (slower, little gain for template text); OpenAI embeddings (paid, online)
Vector database	ChromaDB (persistent, HNSW, cosine)	runs locally with no server; metadata 'where' filters; item id = record_id so hits map to real rows	FAISS (needs a separate id map and no metadata filtering)
Keyword search	own BM25 (k1 1.5, b 0.75) as a sparse matrix	exact condition names and places; persisted .npz loads in seconds	rank_bm25 (rebuilds in memory at start-up)
ML	scikit-learn HistGradientBoosting	accurate, fast, handles missing inputs natively	linear regression, random forest (trained and compared)
API	FastAPI + Uvicorn + Pydantic	typed validation, automatic OpenAPI docs, structured errors	Flask
UI	Streamlit (+ optional React)	required by the guideline; quick, Python-only; React shows a production-style client of the same API	—
Evaluation	pytest, DeepEval	unit/integration proofs + LLM-judged RAG metrics	RAGAS (see section 6)

4. Retrieval strategy

4.1 Chunking strategy

The dataset is tabular, so there are no long documents to split. Each record is one chunk: a deterministic text built only from that record's fields and its derived features. Two versions are produced:

- `search_text` — full description including all metrics (rounded to two decimals); indexed by BM25 and shown to users and the LLM.
- `embedding_text` — the qualitative part (conditions, category, severity, priority, weather, season, issue flag) without numbers; embedded. Numbers are left out because the embedding model reads at most 256 word pieces and captures numeric closeness poorly; numbers are handled by metric similarity instead. 100,000 records share 27,117 distinct embedding texts, each embedded once.

Real chunk for ROW-000001:

```
Network record ROW-098813 (description generated from dataset fields).
Derived conditions: low handover success (low_handover), high fault occurrence rate (high_fault_rate).
Derived category: Mobility / Handover (secondary: Fault / Alarm).
Derived severity: MEDIUM (score 23.1). Derived priority: P3.
Location: Hernandezland, PR (Puerto Rico). Season: Winter. Weather: Rain.
Network metrics: cell availability 98.53%; throughput 430.00 Mbps; latency 120.16 ms; packet loss 4.16%;
    call drop rate 0.29%; handover success 9 …
```

4.2 Embedding approach

all-MiniLM-L6-v2 vectors, L2-normalised so cosine similarity is a dot product; stored in ChromaDB collection network_records with metadata (state, city, season, weather, category, severity, priority, issue flag and 18 condition flags) used for filtering.

4.3 Retrieval method and hybrid search

1. **Query understanding** (rules): adverse/favourable/normal statements per metric, stated values and comparisons, context (season, weather, state). Good/normal phrases are removed before semantic search.
2. **Filters** are validated and applied as a mask and as a Chroma where clause.
3. **Candidates**: the top 300 from each method — semantic (Chroma), keyword (BM25) and metric similarity.
4. **Exact re-scoring** of every candidate with all methods; semantic and keyword scores are min-max normalised over the candidates.
5. **Weighted sum**: semantic 0.6, keyword 0.4, metric 0.6, re-normalised to the active methods; minus 0.15 per condition the user called good.
6. **Final guarantee**: every result is checked against the filters again.

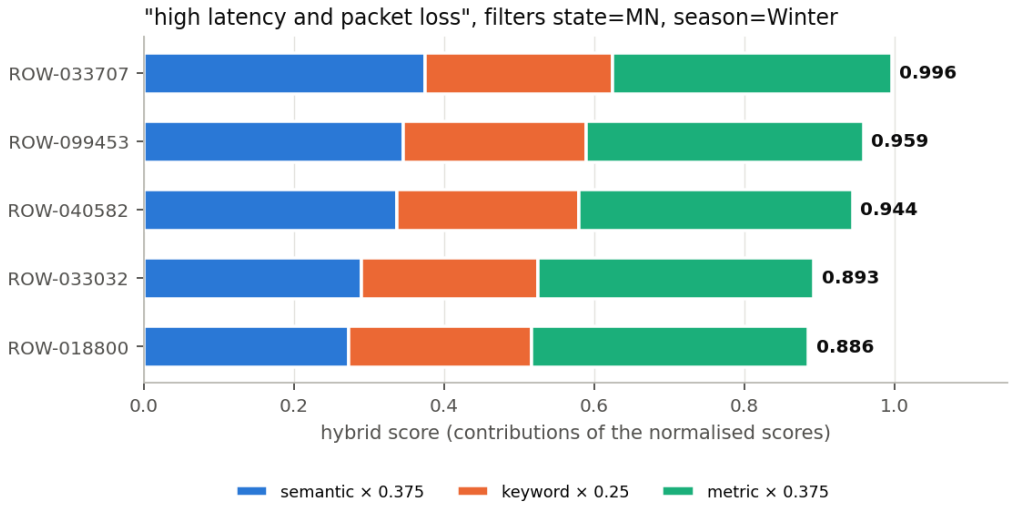


Figure 4.1 — Real hybrid scores for 'high latency and packet loss', Minnesota in winter.

Why hybrid: each method has a blind spot. Embeddings confuse 'good signal' with 'weak signal' (cosine 0.77 for those phrases); BM25 matches only literal words; metric similarity understands numbers but not text. Combining them gave precision@5 of 1.00 on all eight evaluation cases.

5. Agent design

A fixed, linear workflow of six agents on one typed shared state (`AnalysisState`). There are no autonomous loops: each agent runs once, declares the inputs it requires, adds its result and records a timed trace entry. A test proves the agents return exactly the same result as the single-pipeline implementation.

Agent	Role	Reads	Adds
Retrieval	query understanding, filters, hybrid search, evidence	query, filters, metrics	retrieval, evidence
Classification	conditions → category and severity; ML cross-check	evidence	classification
Risk	fault-risk prediction; priority	evidence, classification	risk, priority
Diagnosis	playbook hypotheses with heuristic confidence	evidence	root_cause
Recommendation	actions, checks, monitoring; LLM explanation + grounding gate	root_cause, evidence	recommendations, narrative
Validation	deterministic checks + LLM judge; provenance; report	all of the above	validation, provenance, report_text



Agent order and measured duration for the Phase 30 query (offline mode). With the LLM enabled the Recommendation and Validation agents add LLM calls (~10-40 s).

Figure 5.1 — Agent workflow with measured timings (offline mode).

Communication. Agents never call each other directly; they communicate only through the shared state, which makes every step inspectable and testable. The orchestrator

(`src/agents/orchestrator.py`) runs guardrails first, then the agents in order, and returns the state with the trace. Errors are raised as AgentError and returned by the API as a structured 500.

LLM inside the workflow. The Recommendation agent sends only the evidence package to the LLM, which must fill a fixed JSON schema (summary, diagnosis, one note per hypothesis, limitations). A deterministic grounding gate rejects unsupported numbers, unretrieved record IDs, protocol terms absent from the evidence (X2, RACH, AMF …) and guaranteed outcomes; one retry, then a template. The Validation agent adds an LLM judge; a check passes only if code and judge agree.

6. Evaluation strategy

Layer	Tool	What is measured
Unit and integration tests	pytest (222 tests)	every module; the five required proofs: no raw value fabricated, filters never violated, retrieved IDs are real, LLM cannot add unsupported numbers, no target leakage
Evaluation set	8 hand-made cases (tests/evaluation/eval_cases.json)	condition recall, retrieval precision, average precision, category match, hypothesis in top 2, grounded explanation
RAG quality	DeepEval with the NVIDIA LLM as judge	faithfulness, answer relevancy, contextual precision, contextual relevancy
ML	held-out 20% test set, 5-fold CV	R^2, MAE, RMSE; accuracy, macro precision/recall/F1
Web UI	Vitest + browser runs	formatting, filters, validation; every page end to end

RAGAS. RAGAS covers the same RAG dimensions (faithfulness, context precision/recall, answer relevancy) as DeepEval. Running both with the same judge model would duplicate the measurement, so DeepEval was chosen; RAGAS is listed as optional in the guideline. Results are in docs/Evaluation_Report.pdf.

7. Challenges and trade-offs

Challenge / decision	Choice made	Trade-off
No incident labels in the data	derive category/severity/priority with documented rules	labels are interpretations, always called derived
Industry KPI targets flag 80–88% of rows	worst-decile (P10/P90) thresholds	relative to this dataset, not absolute
Fault rate is a near-formula of 7 columns (R^2 0.9998)	exclude them (served R^2 0.885)	lower score, but honest and usable before a fault
LLMs invent numbers and causes	LLM only explains; grounding gate; template fallback	explanations are sometimes the plainer template
Grounding check is value-based	add an LLM judge	a smaller judge model (gpt-oss-20b) can raise false alarms
Embeddings miss negation and numbers	hybrid with BM25 and metric similarity	one weight tuned on the development set
pandas changed ~15% of floats in the last digit	exact float() parsing + equality test	slower load, exact values
Agent frameworks add complexity	plain linear orchestrator	no dynamic re-planning
Guideline asks for Streamlit; a polished UI was wanted	Streamlit primary + optional React	two front ends to maintain (both use the same API)

Limitations. The dataset is synthetic and has no timestamps, cell IDs, neighbour or transport counters, so causes remain hypotheses and no time-series forecasting is possible. Confidence is a heuristic, not a calibrated probability. The evaluation set is small and was used during development.

Future work. Real operator data with per-cell baselines; transport/neighbour/signalling counters to confirm hypotheses; confidence calibrated from engineer feedback; a held-out benchmark grown from reviewed feedback.